
INFORME DE OPERACIONES

Modelo Entidad–Relación (MER)

Solo Todo

Plataforma de Comparación y Agregación (3 Zonas)

Descripción de entidades y relaciones del diagrama
MER.pdf

1. Introducción

El presente informe documenta las operaciones de diseño del Modelo Entidad-Relación (MER) de SoloTodo, una plataforma de comparación y agregación de productos y servicios. El modelo se organiza en tres grandes zonas funcionales:

- (1) el catálogo y comparación de **PRODUCTOS** junto con sus tiendas y ofertas,
- (2) el catálogo y comparación de **SERVICIOS** junto con sus proveedores y ofertas, y
- (3) los módulos transversales de usuarios, ubicaciones geográficas, búsqueda con inteligencia artificial y obtención de datos mediante web scraping.

Para cada entidad se describe su finalidad dentro del sistema, la clave primaria (**PK**) que la identifica de forma única, las claves foráneas (**FK**) que solicita de otras entidades, y la cardinalidad de cada conexión (1:N, N:1, parent-child o 1:0..1), siguiendo el mismo criterio de análisis usado para **PRODUCT_SPECIFICATION**: entidad que posee su propia PK y que solicita, como FK, la PK de **PRODUCT** y la PK de **CATEGORY_SPECIFICATION**, resolviendo una relación de muchos a uno (N:1) hacia cada una de ellas.

El informe se organiza en diez zonas temáticas que agrupan entidades relacionadas funcionalmente, replicando la estructura visual del diagrama adjunto (*Copia_de_Copia_de_Mejorado_MER.pdf*).

2. Resumen de zonas del modelo

La siguiente tabla resume la cobertura de entidades por zona funcional.

Zona	N.º	Entidades incluidas
Usuarios y Roles	2	ROLE, USER_ACCOUNT
Catálogo de Productos	7	PRODUCT_TYPE, PRODUCT_CATEGORY, BRAND, PRODUCT, PRODUCT_IMAGE, CATEGORY_SPECIFICATION, PRODUCT_SPECIFICATION
Tiendas y Ofertas de Productos	3	STORE, PRODUCT_OFFER, PRICE_HISTORY
Interacción de Usuarios con Productos	5	PRODUCT_COMPARISON, PRODUCT_COMPARISON_ITEM, FAVORITE_PRODUCT, WATCHED_PRODUCT, PRODUCT_REVIEW
Catálogo de Servicios	5	SERVICE_TYPE, SERVICE_CATEGORY, SERVICE, SERVICE_SPECIFICATION, SERVICE_ATTRIBUTE
Proveedores y Ofertas de Servicios	3	PROVIDER, SERVICE_OFFER, SERVICE_PRICE_HISTORY
Interacción de Usuarios con Servicios	5	SERVICE_COMPARISON, SERVICE_COMPARISON_ITEM, FAVORITE_SERVICE, WATCHED_SERVICE, SERVICE_REVIEW
Ubicaciones Geográficas	3	LOCATION, STORE_LOCATION, PROVIDER_LOCATION
Búsqueda y Procesamiento con IA	3	SEARCH_QUERY, SEARCH_QUERY_FILTER, AI_QUERY_PROCESS
Fuentes de Datos y Web Scraping	4	DATA_SOURCE, SCRAPER_CONFIG, SCRAPE_RUN, SCRAPED_DATA

3. Descripción detallada de entidades y conexiones

Zona 1 • Usuarios y Roles

Módulo base que define quién accede a la plataforma y con qué nivel de privilegios. Es el punto de partida del resto del modelo, ya que casi todas las zonas dependen de `USER_ACCOUNT` para registrar la actividad de cada persona usuaria.

ROLE

Tiene la finalidad de catalogar los perfiles de acceso disponibles en la plataforma (por ejemplo, administrador, usuario estándar, moderador), poseyendo su propia PK (`id`).

- Se relaciona con `USER_ACCOUNT` de manera **1:N**, ya que un rol puede ser asignado a muchas cuentas de usuario, pero cada cuenta posee un único rol activo.

USER_ACCOUNT

Tiene la finalidad de almacenar los datos de autenticación e identificación de cada persona que usa la plataforma, poseyendo su propia PK (`id`), en la cual se solicita la PK de `ROLE` como FK (`role_id`).

- Se relaciona con `ROLE` de manera **N:1**, dado que muchas cuentas comparten el mismo rol.
- Es además el origen de múltiples relaciones **1:N** hacia `PRODUCT_COMPARISON`, `FAVORITE_PRODUCT`, `WATCHED_PRODUCT`, `PRODUCT_REVIEW`, `SERVICE_COMPARISON`, `FAVORITE_SERVICE`, `WATCHED_SERVICE`, `SERVICE_REVIEW` y `SEARCH_QUERY`, puesto que una misma persona usuaria puede generar múltiples registros de cada uno de estos tipos.

Zona 2 • Catálogo de Productos

Conjunto de entidades que estructuran la taxonomía y ficha técnica de los productos comparables dentro de la plataforma.

PRODUCT_TYPE

Tiene la finalidad de definir las grandes familias de productos que existen en el sistema (por ejemplo, tecnología, electrodomésticos), poseyendo su propia PK (`id`).

- Se relaciona con `PRODUCT_CATEGORY` de manera **1:N**, ya que un tipo de producto agrupa muchas categorías.
- Se relaciona con `PRODUCT_COMPARISON` de manera **1:N**, dado que las comparaciones que arma un usuario quedan asociadas a un tipo de producto específico.

PRODUCT_CATEGORY

Tiene la finalidad de organizar los productos en categorías jerárquicas, poseyendo su propia PK (`id`), en la cual se solicita la PK de `PRODUCT_TYPE` como FK (`product_type_id`) y la PK de sí misma como FK (`parent_category_id`) para representar subcategorías.

- Se relaciona con `PRODUCT_TYPE` de manera **N:1**, puesto que cada categoría pertenece a un único tipo de producto.
- Se relaciona consigo misma de manera **parent-child (1:N)**, de modo que una categoría padre puede tener muchas categorías hijas, permitiendo árboles de categorías anidadas.
- Se relaciona con `PRODUCT` de manera **1:N**, ya que una categoría agrupa muchos productos.
- Se relaciona con `CATEGORY_SPECIFICATION` de manera **1:N**, porque una categoría define muchas fichas de especificación posibles para sus productos.

BRAND

Tiene la finalidad de registrar las marcas comerciales de los productos, poseyendo su propia PK (`id`).

- Se relaciona con `PRODUCT` de manera **1:N**, ya que una marca puede fabricar o comercializar muchos productos distintos.

PRODUCT

Tiene la finalidad de representar la ficha maestra de cada producto comparable, poseyendo su propia PK (`id`), en la cual se solicita la PK de `PRODUCT_CATEGORY` como FK (`category_id`) y la PK de `BRAND` como FK (`brand_id`).

- Se relaciona con `PRODUCT_CATEGORY` y `BRAND` de manera **N:1**, puesto que cada producto pertenece a una única categoría y a una única marca.
- Es el lado “1” de múltiples relaciones **1:N** hacia `PRODUCT_IMAGE`, `PRODUCT_SPECIFICATION`, `PRODUCT_OFFER`, `PRODUCT_COMPARISON_ITEM`, `FAVORITE_PRODUCT`, `WATCHED_PRODUCT` y `PRODUCT_REVIEW`, ya que un mismo producto puede tener muchas imágenes, especificaciones, ofertas, apariciones en comparaciones, marcas de favorito, seguimientos de precio y reseñas.

PRODUCT_IMAGE

Tiene la finalidad de almacenar las imágenes asociadas a la galería de un producto, poseyendo su propia PK (`id`), en la cual se solicita la PK de `PRODUCT` como FK (`product_id`).

- Se relaciona con `PRODUCT` de manera **N:1**, ya que muchas imágenes pertenecen a un mismo producto.

CATEGORY_SPECIFICATION

Tiene la finalidad de definir qué atributos técnicos son comparables para los productos de una categoría (por ejemplo, RAM, pulgadas de pantalla), poseyendo su propia PK (`id`), en la cual se solicita la PK de `PRODUCT_CATEGORY` como FK (`category_id`).

- Se relaciona con `PRODUCT_CATEGORY` de manera **N:1**, dado que muchas especificaciones pueden pertenecer a una misma categoría.
- Se relaciona con `PRODUCT_SPECIFICATION` de manera **1:N**, ya que una misma definición de especificación puede tomar valores en muchos productos distintos.

PRODUCT_SPECIFICATION

Tiene la finalidad de guardar el valor concreto que toma una especificación para un producto en particular, poseyendo su propia PK (**id**), en la cual se solicita la PK de **PRODUCT** y la PK de **CATEGORY_SPECIFICATION**, ambas como FK (**product_id**, **specification_id**).

- Se relaciona con **PRODUCT** de manera **N:1**, ya que un producto posee muchas especificaciones asociadas.
- Se relaciona con **CATEGORY_SPECIFICATION** de manera **N:1**, dado que una misma especificación definida a nivel de categoría es reutilizada por muchos productos.

Zona 3 • Tiendas y Ofertas de Productos

Módulo que conecta el catálogo de productos con el mercado real, registrando quién vende cada producto, a qué precio y con qué historial.

STORE

Tiene la finalidad de registrar a las tiendas o retailers que publican ofertas de productos, poseyendo su propia PK (**id**).

- Se relaciona con **PRODUCT_OFFER** de manera **1:N**, ya que una tienda publica muchas ofertas.
- Se relaciona con **STORE_LOCATION** de manera **1:N**, porque una misma tienda puede tener presencia en varias ubicaciones físicas.

PRODUCT_OFFER

Tiene la finalidad de representar la publicación de un producto específico en una tienda específica, con su precio y condiciones, poseyendo su propia PK (**id**), en la cual se solicita la PK de **PRODUCT** y la PK de **STORE**, ambas como FK (**product_id**, **store_id**).

- Se relaciona con **PRODUCT** y con **STORE** de manera **N:1**, puesto que un producto puede tener muchas ofertas y una tienda puede publicar muchas ofertas.
- Se relaciona con **PRICE_HISTORY** de manera **1:N**, ya que una misma oferta acumula muchos registros históricos de precio a lo largo del tiempo.

PRICE_HISTORY

Tiene la finalidad de dejar trazabilidad de la variación de precio de una oferta en el tiempo, poseyendo su propia PK (**id**), en la cual se solicita la PK de **PRODUCT_OFFER** como FK (**product_offer_id**).

- Se relaciona con **PRODUCT_OFFER** de manera **N:1**, dado que una oferta genera muchos puntos históricos de precio.

Zona 4 • Interacción de Usuarios con Productos

Entidades que capturan cómo cada usuario utiliza la plataforma sobre productos: comparar, marcar como favorito, seguir precios y opinar.

PRODUCT_COMPARISON

Tiene la finalidad de agrupar un conjunto de productos que un usuario decide comparar entre sí, poseyendo su propia PK (id), en la cual se solicita la PK de USER_ACCOUNT y la PK de PRODUCT_TYPE, ambas como FK (user_id, product_type_id).

- Se relaciona con USER_ACCOUNT y con PRODUCT_TYPE de manera N:1, ya que un usuario puede crear muchas comparaciones y un tipo de producto puede estar presente en muchas comparaciones.
- Se relaciona con PRODUCT_COMPARISON_ITEM de manera 1:N, porque una comparación agrupa muchos ítems (productos) comparados.

PRODUCT_COMPARISON_ITEM

Tiene la finalidad de representar cada producto individual dentro de una comparación, indicando su posición u orden, poseyendo su propia PK (id), en la cual se solicita la PK de PRODUCT_COMPARISON y la PK de PRODUCT, ambas como FK (comparison_id, product_id).

- Se relaciona con PRODUCT_COMPARISON y con PRODUCT de manera N:1, dado que una comparación contiene muchos ítems y un producto puede aparecer en muchas comparaciones distintas.

FAVORITE_PRODUCT

Tiene la finalidad de registrar qué productos ha marcado un usuario como favoritos, poseyendo su propia PK (id), en la cual se solicita la PK de USER_ACCOUNT y la PK de PRODUCT, ambas como FK (user_id, product_id).

- Se relaciona con USER_ACCOUNT y con PRODUCT de manera N:1, ya que un usuario puede marcar muchos productos como favoritos y un producto puede ser favorito de muchos usuarios.

WATCHED_PRODUCT

Tiene la finalidad de permitir que un usuario siga la evolución de precio de un producto y defina un precio objetivo de notificación, poseyendo su propia PK (id), en la cual se solicita la PK de USER_ACCOUNT y la PK de PRODUCT, ambas como FK (user_id, product_id).

- Se relaciona con USER_ACCOUNT y con PRODUCT de manera N:1, puesto que un usuario puede seguir muchos productos y un producto puede ser seguido por muchos usuarios.

PRODUCT_REVIEW

Tiene la finalidad de almacenar las reseñas y calificaciones que los usuarios dejan sobre un producto, poseyendo su propia PK (id), en la cual se solicita la PK de USER_ACCOUNT y la PK de PRODUCT, ambas como FK (user_id, product_id).

- Se relaciona con `USER_ACCOUNT` y con `PRODUCT` de manera **N:1**, ya que un usuario puede escribir muchas reseñas y un producto puede recibir muchas reseñas de distintos usuarios.

Zona 5 • Catálogo de Servicios

Réplica estructural de la zona de productos, pero orientada a servicios (por ejemplo, planes de telefonía, internet o seguros), manteniendo la misma lógica de tipos, categorías y especificaciones.

SERVICE_TYPE

Tiene la finalidad de definir las grandes familias de servicios ofrecidos en la plataforma, poseyendo su propia PK (`id`).

- Se relaciona con `SERVICE_CATEGORY` de manera **1:N**, ya que un tipo de servicio agrupa muchas categorías.
- Se relaciona con `SERVICE_COMPARISON` de manera **1:N**, porque las comparaciones de servicios quedan asociadas a un tipo de servicio específico.

SERVICE_CATEGORY

Tiene la finalidad de organizar los servicios en categorías jerárquicas, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SERVICE_TYPE` como FK (`service_type_id`) y la PK de sí misma como FK (`parent_category_id`).

- Se relaciona con `SERVICE_TYPE` de manera **N:1**, dado que cada categoría pertenece a un único tipo de servicio.
- Se relaciona consigo misma de manera **parent-child (1:N)**, permitiendo subcategorías anidadas.
- Se relaciona con `SERVICE` de manera **1:N**, ya que una categoría agrupa muchos servicios.
- Se relaciona con `SERVICE_SPECIFICATION` de manera **1:N**, porque una categoría define muchas especificaciones posibles para sus servicios.

SERVICE

Tiene la finalidad de representar la ficha maestra de cada servicio comparable, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SERVICE_CATEGORY` como FK (`category_id`).

- Se relaciona con `SERVICE_CATEGORY` de manera **N:1**, ya que cada servicio pertenece a una única categoría.
- Es el lado “1” de las relaciones **1:N** hacia `SERVICE_ATTRIBUTE`, `SERVICE_OFFER`, `SERVICE_COMPARISON_ITEM`, `FAVORITE_SERVICE`, `WATCHED_SERVICE` y `SERVICE_REVIEW`, ya que un mismo servicio puede tener muchos atributos, ofertas, apariciones en comparaciones, marcas de favorito, seguimientos y reseñas.

SERVICE_SPECIFICATION

Tiene la finalidad de definir qué atributos técnicos son comparables para los servicios de una categoría, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SERVICE_CATEGORY` como FK (`category_id`).

- Se relaciona con `SERVICE_CATEGORY` de manera **N:1**, dado que muchas especificaciones pueden pertenecer a una misma categoría.
- Se relaciona con `SERVICE_ATTRIBUTE` de manera **1:N**, ya que una misma definición de especificación puede tomar valores en muchos servicios distintos.

SERVICE_ATTRIBUTE

Tiene la finalidad de guardar el valor concreto que toma una especificación para un servicio en particular, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SERVICE` y la PK de `SERVICE_SPECIFICATION`, ambas como FK (`service_id`, `specification_id`).

- Se relaciona con `SERVICE` de manera **N:1**, ya que un servicio posee muchos atributos asociados.
- Se relaciona con `SERVICE_SPECIFICATION` de manera **N:1**, dado que una misma especificación es reutilizada por muchos servicios.

Zona 6 • Proveedores y Ofertas de Servicios

Módulo equivalente a Tiendas y Ofertas, pero para el mundo de servicios, incorporando conceptos propios como periodicidad de facturación y costos de instalación.

PROVIDER

Tiene la finalidad de registrar a las empresas o proveedores que publican ofertas de servicios, poseyendo su propia PK (`id`).

- Se relaciona con `SERVICE_OFFER` de manera **1:N**, ya que un proveedor publica muchas ofertas de servicio.
- Se relaciona con `PROVIDER_LOCATION` de manera **1:N**, porque un mismo proveedor puede tener cobertura en varias ubicaciones.

SERVICE_OFFER

Tiene la finalidad de representar la publicación de un servicio específico por parte de un proveedor específico, con su tarifa y condiciones contractuales, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SERVICE` y la PK de `PROVIDER`, ambas como FK (`service_id`, `provider_id`).

- Se relaciona con `SERVICE` y con `PROVIDER` de manera **N:1**, puesto que un servicio puede tener muchas ofertas y un proveedor puede publicar muchas ofertas.
- Se relaciona con `SERVICE_PRICE_HISTORY` de manera **1:N**, ya que una misma oferta acumula muchos registros históricos de precio.

SERVICE_PRICE_HISTORY

Tiene la finalidad de dejar trazabilidad de la variación tarifaria de una oferta de servicio en el tiempo, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SERVICE_OFFER` como FK (`service_offer_id`).

- Se relaciona con `SERVICE_OFFER` de manera **N:1**, dado que una oferta genera muchos puntos históricos de precio.

Zona 7 • Interacción de Usuarios con Servicios

Espejo funcional de la Zona 4, pero orientado a servicios: comparar, marcar favoritos, seguir tarifas y opinar sobre proveedores.

SERVICE_COMPARISON

Tiene la finalidad de agrupar un conjunto de servicios que un usuario decide comparar entre sí, poseyendo su propia PK (`id`), en la cual se solicita la PK de `USER_ACCOUNT` y la PK de `SERVICE_TYPE`, ambas como FK (`user_id`, `service_type_id`).

- Se relaciona con `USER_ACCOUNT` y con `SERVICE_TYPE` de manera **N:1**, ya que un usuario puede crear muchas comparaciones y un tipo de servicio puede estar presente en muchas comparaciones.
- Se relaciona con `SERVICE_COMPARISON_ITEM` de manera **1:N**, porque una comparación agrupa muchos ítems (servicios) comparados.

SERVICE_COMPARISON_ITEM

Tiene la finalidad de representar cada servicio individual dentro de una comparación, indicando su posición, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SERVICE_COMPARISON` y la PK de `SERVICE`, ambas como FK (`comparison_id`, `service_id`).

- Se relaciona con `SERVICE_COMPARISON` y con `SERVICE` de manera **N:1**, dado que una comparación contiene muchos ítems y un servicio puede aparecer en muchas comparaciones distintas.

FAVORITE_SERVICE

Tiene la finalidad de registrar qué servicios ha marcado un usuario como favoritos, poseyendo su propia PK (`id`), en la cual se solicita la PK de `USER_ACCOUNT` y la PK de `SERVICE`, ambas como FK (`user_id`, `service_id`).

- Se relaciona con `USER_ACCOUNT` y con `SERVICE` de manera **N:1**, ya que un usuario puede marcar muchos servicios como favoritos y un servicio puede ser favorito de muchos usuarios.

WATCHED_SERVICE

Tiene la finalidad de permitir que un usuario siga la evolución tarifaria de un servicio y defina un precio objetivo de notificación, poseyendo su propia PK (`id`), en la cual se solicita la PK de `USER_ACCOUNT` y la PK de `SERVICE`, ambas como FK (`user_id`, `service_id`).

- Se relaciona con `USER_ACCOUNT` y con `SERVICE` de manera **N:1**, puesto que un usuario puede seguir muchos servicios y un servicio puede ser seguido por muchos usuarios.

SERVICE_REVIEW

Tiene la finalidad de almacenar las reseñas y calificaciones que los usuarios dejan sobre un servicio, poseyendo su propia PK (**id**), en la cual se solicita la PK de **USER_ACCOUNT** y la PK de **SERVICE**, ambas como FK (**user_id**, **service_id**).

- Se relaciona con **USER_ACCOUNT** y con **SERVICE** de manera **N:1**, ya que un usuario puede escribir muchas reseñas y un servicio puede recibir muchas reseñas de distintos usuarios.

Zona 8 • Ubicaciones Geográficas

Entidades transversales que permiten geolocalizar tanto tiendas como proveedores y enriquecer las búsquedas con criterios espaciales.

LOCATION

Tiene la finalidad de catalogar ubicaciones geográficas normalizadas (país, región, ciudad, comuna, coordenadas), poseyendo su propia PK (**id**).

- Se relaciona con **STORE_LOCATION** de manera **1:N**, ya que una misma ubicación puede alojar varias tiendas.
- Se relaciona con **PROVIDER_LOCATION** de manera **1:N**, ya que una misma ubicación puede alojar varios proveedores.
- Se relaciona con **SEARCH_QUERY** de manera **1:N**, dado que muchas búsquedas pueden originarse o filtrarse por una misma ubicación.

STORE_LOCATION

Tiene la finalidad de vincular una tienda con una dirección física concreta, poseyendo su propia PK (**id**), en la cual se solicita la PK de **STORE** y la PK de **LOCATION**, ambas como FK (**store_id**, **location_id**).

- Se relaciona con **STORE** y con **LOCATION** de manera **N:1**, puesto que una tienda puede tener muchas sucursales y una ubicación puede alojar muchas tiendas.

PROVIDER_LOCATION

Tiene la finalidad de vincular un proveedor de servicios con una zona de cobertura o dirección concreta, poseyendo su propia PK (**id**), en la cual se solicita la PK de **PROVIDER** y la PK de **LOCATION**, ambas como FK (**provider_id**, **location_id**).

- Se relaciona con **PROVIDER** y con **LOCATION** de manera **N:1**, ya que un proveedor puede cubrir muchas ubicaciones y una ubicación puede ser cubierta por muchos proveedores.

Zona 9 • Búsqueda y Procesamiento con Inteligencia Artificial

Módulo que registra las consultas realizadas por los usuarios y el procesamiento inteligente (NLP/IA) que interpreta la intención de búsqueda.

SEARCH_QUERY

Tiene la finalidad de registrar cada búsqueda realizada por un usuario, incluyendo el texto original, filtros de precio y estado del procesamiento, poseyendo su propia PK (`id`), en la cual se solicita la PK de `USER_ACCOUNT` y la PK de `LOCATION`, ambas como FK (`user_id`, `location_id`).

- Se relaciona con `USER_ACCOUNT` y con `LOCATION` de manera **N:1**, ya que un usuario puede realizar muchas búsquedas y una ubicación puede estar asociada a muchas búsquedas.
- Se relaciona con `SEARCH_QUERY_FILTER` de manera **1:N**, porque una búsqueda puede tener muchos filtros aplicados.
- Se relaciona con `AI_QUERY_PROCESS` de manera **1:0..1**, ya que una búsqueda puede tener, como máximo, un único registro de procesamiento por IA asociado (o ninguno, si no fue procesada).

SEARCH_QUERY_FILTER

Tiene la finalidad de almacenar cada filtro adicional aplicado a una búsqueda (nombre y valor del filtro), poseyendo su propia PK (`id`), en la cual se solicita la PK de `SEARCH_QUERY` como FK (`search_query_id`).

- Se relaciona con `SEARCH_QUERY` de manera **N:1**, dado que una búsqueda puede tener muchos filtros, pero cada filtro pertenece a una sola búsqueda.

AI_QUERY_PROCESS

Tiene la finalidad de registrar el resultado del análisis de intención (NLP) sobre una búsqueda, incluyendo entidad detectada, JSON estructurado y tiempo de respuesta, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SEARCH_QUERY` como FK (`search_query_id`).

- Se relaciona con `SEARCH_QUERY` de manera **1:1 (0..1:1)**, ya que cada búsqueda procesada por IA genera, como máximo, un único registro de procesamiento.

Zona 10 • Fuentes de Datos y Web Scraping

Módulo de ingesta de información: define de dónde provienen los datos, cómo se extraen automáticamente y qué se obtiene en cada ejecución.

DATA_SOURCE

Tiene la finalidad de catalogar las fuentes externas de datos (tiendas, proveedores, portales) que se pueden scrapear, poseyendo su propia PK (`id`).

- Se relaciona con `SCRAPER_CONFIG` de manera **1:N**, ya que una misma fuente de datos puede tener configurados varios scrapers (por ejemplo, uno por tipo de entidad).

SCRAPER_CONFIG

Tiene la finalidad de definir los parámetros técnicos de ejecución de un scraper (tipo, objetivo, frecuencia, límite de tasa), poseyendo su propia PK (`id`), en la cual se solicita la PK de `DATA_SOURCE` como FK (`data_source_id`).

- Se relaciona con `DATA_SOURCE` de manera **N:1**, dado que muchas configuraciones de scraping pueden apuntar a la misma fuente.
- Se relaciona con `SCRAPE_RUN` de manera **1:N**, porque una misma configuración se ejecuta muchas veces a lo largo del tiempo.

SCRAPE_RUN

Tiene la finalidad de registrar cada ejecución concreta de un scraper, con sus métricas de resultado (encontrados, procesados, fallidos), poseyendo su propia PK (`id`), en la cual se solicita la PK de `SCRAPER_CONFIG` como FK (`scraper_config_id`).

- Se relaciona con `SCRAPER_CONFIG` de manera **N:1**, ya que una configuración genera muchas ejecuciones.
- Se relaciona con `SCRAPED_DATA` de manera **1:N**, porque una sola ejecución puede recolectar muchos registros crudos de datos.

SCRAPED_DATA

Tiene la finalidad de almacenar el dato crudo (raw) extraído por el scraper antes de su normalización e integración al catálogo, poseyendo su propia PK (`id`), en la cual se solicita la PK de `SCRAPE_RUN` como FK (`scrape_run_id`).

- Se relaciona con `SCRAPE_RUN` de manera **N:1**, dado que una ejecución produce muchos registros de datos extraídos.

4. Conclusiones

El modelo de SoloTodo replica de forma consistente la misma arquitectura para productos y servicios (catálogo, especificaciones, ofertas, historial de precios, comparaciones, favoritos, seguimientos y reseñas), lo que facilita su mantenimiento y extensión a futuros tipos de entidades comparables.

Las relaciones jerárquicas parent-child en `PRODUCT_CATEGORY` y `SERVICE_CATEGORY` permiten construir árboles de categorías de profundidad variable. La relación **1:0..1** entre `SEARCH_QUERY` y `AI_QUERY_PROCESS` es la única cardinalidad opcional del modelo, reflejando que no todas las búsquedas son procesadas por el motor de inteligencia artificial.

Finalmente, el módulo de scraping (`DATA_SOURCE` → `SCRAPER_CONFIG` → `SCRAPE_RUN` → `SCRAPED_DATA`) actúa como una cadena de ingesta encadenada en cascada **1:N**, que alimenta de datos crudos al resto del sistema antes de su normalización hacia `PRODUCT`, `PRODUCT_OFFER`, `SERVICE` y `SERVICE_OFFER`.